2025 语言课程设计实验报告

学号：09024201

姓名：王晨茜

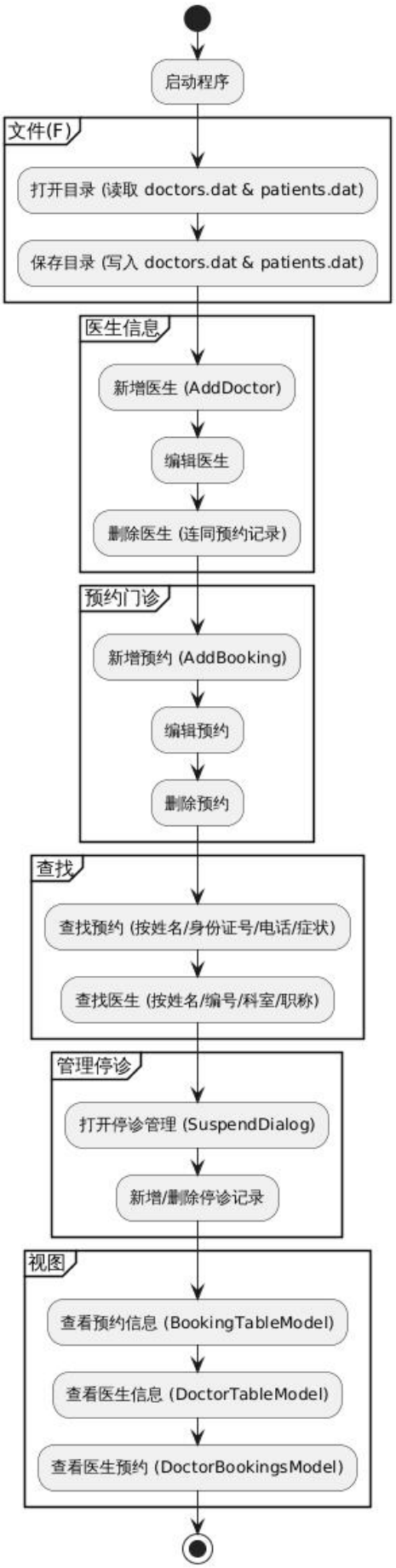
1 题目

医院专家门诊预约管理系统
[要求] 该系统需创建和管理以下信息：
1、门诊专家信息：专家姓名、编号、性别、年龄、
职称、门诊科目、服务时间、门诊预约数据集等；
2、门诊预约信息：病人姓名、身份证
号、性别、年龄、预约时间、联系电话、症状描述等。

系统功能要求如下：
1．创建和管理描述门诊专家信息的对象；
2．创建和管理描述门诊预约信息的对象；
3．门诊专家注册，在门诊专家下添加门诊预约数据，预约身份证号不能重复。
4．门诊专家、门诊预约信息的变更管理等；
5. 基本查询功能；
6．数据文件读写：文件中包含所有门诊专家、门诊预约信息等数据；
7．基本信息显示：1）所有门诊专家信息显示；2）特定门诊专家的门诊预约信息；
8．可选功能提升：管理停诊、预约时间防重复等控制；

2 设计思路

2.1 整体设计思路展示:


启动程序
文件(F)
打开目录 (读取 doctors.dat & patients.dat)
保存目录 (写入 doctors.dat & patients.dat)
医生信息
新增医生 (AddDoctor)
编辑医生
删除医生 (连同预约记录)
预约门诊
新增预约 (AddBooking)
编辑预约
删除预约
查找
查找预约 (按姓名/身份证号/电话/症状)
查找医生 (按姓名/编号/科室/职称)
管理停诊
打开停诊管理 (SuspendDialog)
新增/删除停诊记录
视图
查看预约信息 (BookingTableModel)
查看医生信息 (DoctorTableModel)
查看医生预约 (DoctorBookingsModel)

2.2 C++部分

2.2.1 类的分别叙述

1）Patient 病人 一次预约实体

关键成员：

QString p_name, p_id, p_symptom

int p_sex, p_age, p_phone

QDate p_booking_date

QTime p_booking_time

关键函数 / 职责：

savePatient(QTextStream&) const：逐行文本序列化；

readPatient(QTextStream&)：反序列化

系统定位：

预约的实体，独立存在于全局表，也会作为医生名下副本。

2） Doctor 医生个体

关键成员：

基本信息：d_name, d_id, d_sex, d_age, d_title, d_subject

全局工作时段：QTime d_time_s, d_time_e

周自定义：QTime weekStart[7], weekEnd[7]; bool weekEnabled[7]

名下预约：QVector<Patient> d_patient; int d_pat_num

关键函数 / 职责：

workTimeFor(const QDate&, QTime&, QTime&)：解析某日工作时段

saveDoctor(QTextStream&) const / readDoctor(QTextStream&)：文件内容读写

系统定位：

医生实体，承载排班，管理其名下的预约集合。

3） PatInforTable 预约集合

关键成员：

QVector<Patient> pats

int pats_num

关键函数 / 职责：

readPatFromFile / savePatToFile：读写 patients.dat

addPat / removeAt / updateAt / setPat / replaceAll：增删改

系统定位：

预约的总集合，所有操作必须同步医生名下。

4） DoctorInfortable 医生集合

关键成员

QVector<Doctor> docs

int docs_num

关键函数 / 职责：

readDoctorFromFile / saveDoctorToFile：读写 doctors.dat

addDoctor / removeDoctorAt：管理医生集合

系统定位：

全局医生集合，提供读写与访问

5） BookingTableModel 全院预约表模型

关键成员：

DoctorInfortable* dt_

PatInforTable* pt_

关键函数 / 职责：

rowCount/data/headerData：提供表格数据

patientTimeClash：检测身份证与同一日期下的时间冲突

fitShift：把预约时间对齐到医生上班时段

addBooking/updateBooking/removeBooking：增删改

系统定位：

预约表格的自制模型

桥梁模型，负责全局预约和医生预约之间的一致性与冲突检测。

6） DoctorBookingsTableModel 单医生预约表模型

关键成员：

DoctorInfortable* m_table

int m_docIndex

QVector<Row> m_rows（Row 含 Patient、科室、医生名）

关键函数 / 职责：

refresh：生成并排序

data/headerData：返回行内容

系统定位：

某个特定医生名下预约的明细模型

7） DoctorTableModel 医生表模型

关键成员：

DoctorInfortable* ds_

关键函数 / 职责：

data/headerData：映射医生基础字段

flags/setData：允许年龄和职称修改并回写

系统定位：

医生集合的表格模型

8） Suspend 停诊实体

关键成员：

int doctorIndex

QDate date

QString reason

关键函数 / 职责：

支持存入/读取停诊医生与日期

getter/setter：访问和修改停诊数据

系统定位：

表达医生在某日期停诊的约束

9） SuspendModel 停诊表模型

关键成员：

QVector<Suspend> suspends

关键函数 / 职责：

rowCount/data：表格展示停诊情况

add/remove：修改停诊集合

系统定位：

停诊数据的展示与管理模型

10） SearchTableModel 搜索结果表模型

关键成员：

QVector<Patient> results

关键函数 / 职责：

rowCount/data：返回搜索结果的预约信息

setResults：接受搜索结果并刷新

系统定位：

用于搜索对话框结果展示的模型

11） AddBooking 新建预约

关键成员：

引用/指针：DoctorInfortable* doctors，const QVector<SuspendItem> *m_suspends

暂存体：候选 Patient（姓名、证件、日期、起始时间、电话、症状等）

关键函数：

updateDoctorsComboBox(const QString &department)：按所选科室刷新医生下拉框，保证后续校验基于有效医生集合。

initDepartmentsAndDoctors()：初始化“科室→医生”映射与初始列表。

updateDateTimeRange(const QDate &d)：依据医生的 workTimeFor 动态限定当日可选时间范围。

isSuspended(const QDate &day, int docIndex)：做“当天是否停诊”的业务判断。

resetForm()：清空并回到默认状态

findNearestAvailable(...)：取到就近可用半小时格

ceilToHalfHour(...)：时间向上取到 0/30 分

overlap30min(...)：30 分钟区间重叠判断

getBookingInfo() / validateBookingInfo(...)：打包并校验一次预约输入

系统定位：

新建预约；对齐可预约时段；检查冲突；必要时自动向后寻找可用半小时空档。

12） AddDoctor 新增医生
关键成员：
Doctor tmp_;目标医生
关键函数：
dayEnabled(int), dayStart(int), dayEnd(int)：MainWindow 通过这些函数把周排班写回 Doctor
系统定位：
新增/编辑医生；维护基本信息与周排班。

13） SearchDialog 搜索弹窗
关键成员：
enum SearchType { SearchDoctor, SearchPatient }
void* dataTable（根据 SearchType 解释为 DoctorInfortable* 或 PatInforTable*）
SearchTableModel* model
关键函数 / 职责：
searchDoctors(...)：在 DoctorInfortable 内按条件（姓名/编号/科室/职称）过滤
searchPatients(...)：在 PatInforTable 内按条件（姓名/证件/电话/症状）过滤
setResults(...)：设置到 SearchTableModel（医生结果集或患者结果集两种模式）
系统定位：
执行医生或患者的检索，展示结果。

14） SuspendDialog 停诊弹窗
关键成员：
SuspendModel* model
DoctorInfortable* 医生总表指针
关键函数 / 职责：
onAdd() 新增一条记录
onDel() 删除所选记录
系统定位：
停诊管理，基于 SuspendModel 增删停诊记录

15） MainWindow 系统主界面
关键成员：
仓储：DoctorInfortable doctorTable、PatInforTable patientTable
模型：DoctorTableModel* modelDoctors_、BookingTableModel* modelBookings_、DoctorBookingsModel* modelDocBookings_、PatientTableModel* modelPatients_

关键函数：

on_action_O_triggered()：从文件读取 doctors.dat 和 patients.dat

on_action_S_triggered()：将当前的医生和患者数据写回到用户选择的目录下的 doctors.dat 与 patients.dat 文件。

on_addDoctorButton_clicked()：弹出 AddDoctor 窗口；若用户确认，则调用 doctorTable.addDoctor() 并刷新 DoctorTableModel。

on_editDoctorButton_clicked()：编辑当前选中的医生：调用 AddDoctor 载入医生数据，修改后覆盖 doctorTable.getDoctor(row)，最后刷新模型。

on_removeDoctorButton_clicked()：删除选中的医生。

on_addBookingButton_clicked()：弹出 AddBooking，并传入 doctorTable 和 m_suspends；若确认，取得新 Patient 与医生索引，调用 BookingTableModel::addBooking() 更新全局与医生副本。

on_editBookingButton_clicked()：编辑选中预约

on_removeBookingButton_clicked()：删除选中预约

on_action_Search_triggered()：打开 SearchDialog，根据条件搜索 DoctorInfortable 或 PatInforTable，结果放入 SearchTableModel 展示。

on_action_Suspend_triggered()：打开 SuspendDialog，对 m_suspends 增删停诊项；结果会影响 AddBooking 时的可约判定。

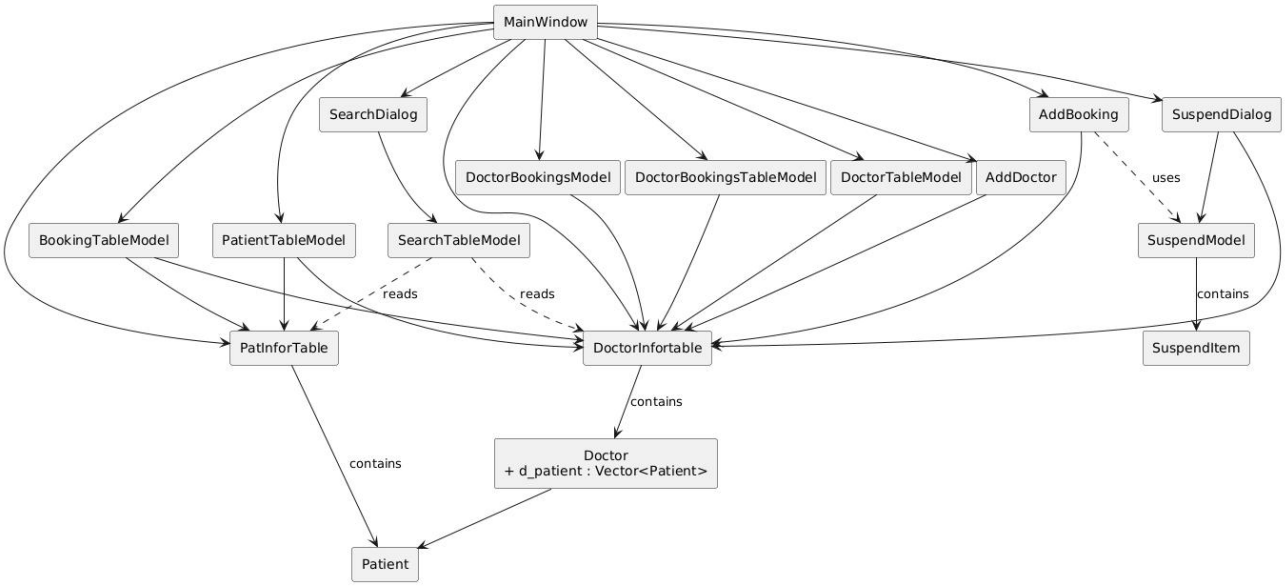
updateStatusBar()：更新状态栏显示，包括医生数、患者预约数、当前文件路径等。

系统定位：

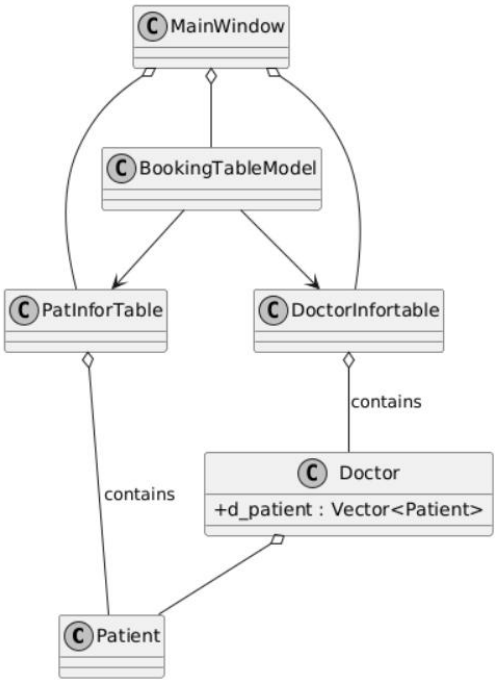
系统主界面与业务聚合

2.2.2 类之间的关系

关系示意图如下图：



其中几个主要类的关系如下



Doctor：有名下预约列表 d_patient 若干 Patient 的副本。
PatInforTable：包含所有 Patient
DoctorInfortable：包含所有 Doctor

BookingTableModel → 依赖 PatInforTable + DoctorInfortable
DoctorBookingsTableModel → 依赖 DoctorInfortable
DoctorTableModel → 依赖 DoctorInfortable，行列访问医生信息

AddDoctor → 读/写 DoctorInfortable 中单个 Doctor。
AddBooking → 读 DoctorInfortable 与停诊集合；确认后由上层把 Patient 写入 PatInforTable，并追加到 Doctor.d_patient。

SearchDialog → 在 PatInforTable 或 DoctorInfortable 内检索，结果交给 SearchTableModel。

SuspendDialog → 操作 SuspendModel（增删停诊项）。

MainWindow 聚合
持有：DoctorInfortable、PatInforTable 与各模型实例。
负责：数据加载/保存；预约新增/编辑/删除时同时刷新模型

2.3 qt 部分

2.3.1 使用的 Qt 框架与模式

Qt Widgets：全部界面基于 QWidget 体系：QMainWindow、QDialog、QTableView 等

Model/View 模式：

自定义 QAbstractTableModel 子类：BookingTableModel、DoctorTableModel、DoctorBookingsModel、DoctorBookingsTableModel、PatientTableModel、SearchTableModel、SuspendModel。

视图主要是 QTableView，由模型驱动刷新，不直接存数据。

2.3.2 信号/槽：

菜单 QAction 触发各业务对话框（新增/编辑/删除/搜索/停诊）。

模型的 dataChanged、layoutChanged、modelReset 驱动表格刷新。

I/O 与文本编码：QFile 、 QTextStream 读写 doctors.dat / patients.dat。

布局系统：QFormLayout、QGridLayout、QVBoxLayout、QHBoxLayout，保证控件自适应。

2.3.3 UI

AddBooking.ui（预约新建/编辑对话框，QDialog）

AddDoctor.ui（医生新增/编辑对话框，QDialog）

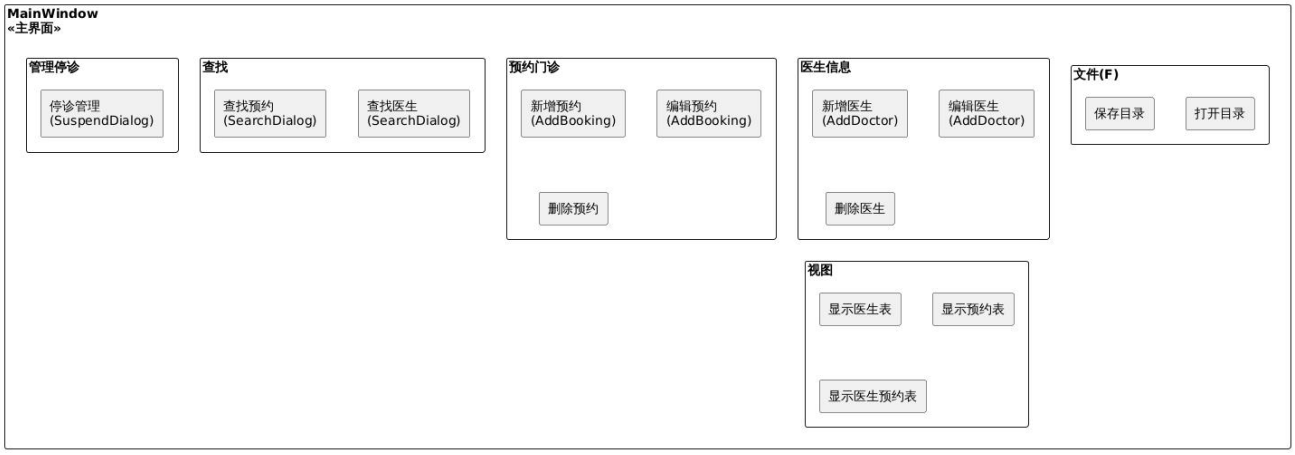
MainWindow.ui（主窗口，QMainWindow）

SearchDialog.ui（检索对话框，QDialog）

SuspendDialog.ui（停诊管理对话框，QDialog）

2.3.4 每个 UI 的控件与功能

整体 ui 显示关系如图



1) AddBooking 预约新建/编辑

布局：QFormLayout

主要控件与用途

lineEdit_name：患者姓名输入。

lineEdit_id：身份证号输入（后续用于同人同日冲突检测、医生反查）。

comboBox_sex：性别选择（男/女）。

spinBox_age：年龄输入。

textEdit_symptom：症状描述（在持久化时会做 Base64）。
comboBox_department：门诊科室选择（驱动医生下拉刷新）。
comboBox_doctor：门诊医生选择（与科室联动）。
dateTimeEdit_time：预约起始时间（半小时吸附；随医生工作时段限定合法范围）。
lineEdit_phone：联系电话。

buttonBox (OK/Cancel)：确认/取消。

对话框内的关键逻辑
updateDoctorsComboBox(department)：按科室筛医生并填充 comboBox_doctor
updateDateTimeRange(date)：读取所选医生当日 workTimeFor()，限制 dateTimeEdit_time 的范围。
isSuspended(date, doctorIdx)：若当日停诊，禁止当前起始时段并提示原因。
findNearestAvailable(...)：若当前半小时时隙冲突/不可用，向后滚动扫描最近可用槽位，并回显。
getBookingInfo()：从输入控件收集 Patient 与 doctorIndex，交给上层保存。

控件之间的关系：
comboBox_department → 驱动 → comboBox_doctor
选择科室后刷新医生列表，只展示该科室的医生；若科室变更，会重置医生选择。
comboBox_doctor + date → 限定 → dateTimeEdit_time 的可选时间范围
根据所选医生的当日工作时段动态设置最早/最晚可选时间
buttonBox.OK → 收集 → 当前控件值
组装为 Patient + 目标医生索引返回给上层；若校验不通过则阻止关闭

2) AddDoctor 医生新增/编辑
布局：外层 QGroupBox + QGridLayout；“默认服务时间”与“每周班次”分区。
主要控件与用途
lineEdit_DName：姓名
lineEdit_DId：编号
comboBox_DSex：性别
spinBox_DAge：年龄
comboBox_DTitle：职称
comboBox_D_subject：门诊科目
timeEdit_DSTime：默认开始时间（写入 Doctor::d_time_s）
timeEdit_DETime：默认结束时间（写入 Doctor::d_time_e）
chkMon：是否启用周一特定时间
timeMonStart：周一开始时间（写入 Doctor::weekStart[1]）
timeMonEnd：周一结束时间（写入 Doctor::weekEnd[1]）

（周二至周日分别对应 chkTue…chkSun / timeTueStart… / timeTueEnd… ）

对话框内的关键逻辑

读取/写回周排班：通过接口 dayEnabled(i) / dayStart(i) / dayEnd(i)（0..6）提供/获取每个工作日的启用与时段；主窗口在“确定”后把这些值写进 Doctor.weekEnabled/weekStart/weekEnd。

默认上下班时段：timeEdit_DSTime、timeEdit_DETime 写入 Doctor.d_time_s/d_time_e。

基本字段填充 Doctor 对象（姓名/编号/性别/年龄/职称/科室）。

控件之间的关系：

每天的 QCheckBox（是否启用） → 控制 → 当天两枚 QTimeEdit 的启用取消勾选时，表示该日不排班；相应时间编辑控件禁用或置灰。

“默认服务时间”与“每周班次”之间的关系：

当某天未启用周班时，系统逻辑回退到“默认服务时间”；勾选启用后，当天以自定义时间为准。

buttonBox.OK → 收集 → 控件值

3) MainWindow 系统主窗口

主要区域与控件

QMenuBar 菜单：

文件(F)：action_O 打开、action_S 保存。

医生信息：actionDoctorAdd / actionDoctorEdit / actionDoctorDelete。

预约门诊：actionBookingAdd / actionBookingEdit / actionBookingDel。

查找：actionSearchBookings 查预约 、actionSearchDoctors 查医生 。

管理停诊：actionManageSuspend。

视图：actionViewBookings 看全院预约 、actionViewDoctors 看医生列表 、actionViewDoctorBookings 看单医生预约。

QStatusBar：显示统计信息：医生数、预约数。

centralWidget：承载表格视图。

主窗口承载的模型/视图

全院预约：QTableView ← BookingTableModel

医生列表：QTableView ← DoctorTableModel。

单医生预约：QTableView ← DoctorBookingsTableModel。

菜单动作将这些视图切换、刷新或弹出子对话框。

控件之间的关系

各 QAction → 触发 → 子对话框 AddDoctor / AddBooking / SearchDialog / SuspendDialog

对话框 Accepted 后，主窗口刷新对应模型与状态栏。

菜单“视图”中的 QAction → 切换 → centralWidget 中当前可见的 QTableView

例如查看“全院预约”时显示与 BookingTableModel 绑定的表格，查看“医生信息”时显示与 DoctorTableModel 绑定的表格。
模型 dataChanged/layoutChanged/modelReset → 联动 → 对应表格自动刷新
打开/保存动作 → 驱动 → 仓储读写
载入后发出模型的 modelReset/layoutChanged；保存成功后更新 statusbar

4) SearchDialog 检索对话框
布局：上方控制面板 + 下方结果表格
主要控件与用途
comboField：字段选择（随搜索对象不同而切换：医生/预约）。
searchEdit：关键字输入框。
searchButton：执行搜索。
tableViewSearch：QTableView 展示结果，模型为 SearchTableModel。
resultCountLabel / searchLabel：回显统计。
closeButton：关闭对话框。

对话框内关键逻辑
设置数据源：传入 DoctorInfortable* 或 PatInforTable*。
点击“搜索”：在对应仓储内过滤，SearchTableModel::setDoctorResults(...) / setPatientResults(...) 填充表格。
字段下拉 comboField 的可选项会随“搜索医生/搜索预约”切换。

控件之间的关系
comboField 选项 → 决定 → 搜索字段 & 数据源
选择医生字段时，数据源为 DoctorInfortable；选择预约字段时，数据源为 PatInforTable。
searchButton / 回车 → 触发 → 过滤逻辑
在对应仓储中线性筛选，结果填入 SearchTableModel，tableViewSearch 自动刷新。
SearchTableModel 更新 → 驱动 → resultCountLabel
结果数量和提示文本更新，反馈给用户

5) SuspendDialog 停诊管理
布局：上方工具条 + 下方结果
主要控件与用途
btnAdd / btnDel / btnClose
tableViewSuspend：QTableView，模型为 SuspendModel。
对话框内关键逻辑
新增/删除停诊记录：直接操作 SuspendModel 的增删接口（内部维护 QVector<SuspendItem>。
关闭时导出集合供 AddBooking 判定当天是否可约

控件之间的关系：

btnAdd/btnDel → 操作 → SuspendModel

模型变更后 tableViewSuspend 即刻刷新；删除按钮通常依赖表格当前选择（无选择时禁用）。

SuspendModel 的数据 → 被 → AddBooking 查询

AddBooking 计算可用时段时，会根据“医生+日期”命中该集合而判定当天不可约。

2.4 程序所完成的功能

2.4.1 医生与预约管理

系统支持医生信息和患者预约的统一管理。可新增、修改、删除医生及其排班，也可新增、修改、删除患者预约。视图功能中，可以显示医生数据表格，预约数据表格以及特定医生下的按时间排序的预约数据表格。

2.4.2 医生信息维护

在医生管理界面，可以录入医生的基本信息（姓名、编号、性别、年龄、职称、科室等），并可设置默认上下班时间及按星期的排班情况。

2.4.3 患者预约管理

在预约界面中，可以录入患者的基本信息（姓名、身份证号、性别、年龄、联系方式、症状），并选择科室、医生及具体预约时间。系统自动检测冲突、对齐半小时时间段，并根据医生排班与停诊情况进行判断。其中，冲突检测使预约只能在界面上输入可行的预约时间（如医生在班且无预约等），而病人层面的冲突检测使同一病人不能同一时间预约两次门诊。

2.4.4 信息修改与删除

医生与预约信息可分别在表格视图和下拉选框中直接编辑、删除，且弹出的编辑框已预填原信息，且固定设置姓名和编号不可改；删除医生时，系统会同步清理该医生的预约记录，保持数据一致。

2.4.5 查询功能

系统提供医生与患者两类检索方式。支持根据姓名、编号、科室、职称等条件查找医生，或根据姓名、身份证号、电话、症状等条件查找患者。查询结果显示在独立界面，并可直接执行修改和删除操作。

2.4.6 停诊管理

系统支持停诊功能，管理员可为某个医生指定停诊日期与原因。停诊信息在预约时会被自动检查，防止患者预约到停诊日期。

2.4.7 数据文件管理

所有医生信息与患者预约均存储在文件中。系统支持以文件夹为单位进行批量读取和保存，目录中包含 doctors.dat 与 patients.dat 两个文件。用

户可通过菜单操作完成数据导入和导出。

2.4.8 细节优化

进行了如下限制：
不同病人身份证号不能重复；
不同医生编号不能重复；
预约时间限制在医生排班时间内；
预约必须按半小时对齐；
冲突时间自动弹开到最近可用时间段

3 文件格式描述（详细描述所保存和读取的文件格式）

3.1 医生信息文件（doctors.dat）

结构：
首行：整数 N，表示医生数量。

接下来每位医生的信息（按顺序写出，每个字段占一行）：
姓名（字符串）
编号（字符串）
性别（整数 0/1）
年龄（整数）
职称（字符串）
科室（字符串）
默认开始时间（格式 hh:mm）
默认结束时间（格式 hh:mm）
每周班次（7 天，每天一行或多行组合，包含启用标志与起止时间，若无则用“-”代替）
名下预约数（整数）
该医生名下所有预约的详细信息（与 patients.dat 中单条记录格式一致）

3.2 预约信息文件（patients.dat）

结构：
首行：整数 M，表示预约数量。

每条预约的信息：
姓名（字符串）
身份证号（字符串）
性别（整数）
年龄（整数）
联系电话（整数或字符串）
预约日期（格式 yyyy-MM-dd）

预约时间（格式 hh:mm，30 分钟）
症状描述

3.3 读取与写入文件夹

由于系统同时管理医生信息与预约信息，因此在数据持久化时，读写两个文件：
医生信息文件：doctors.dat
患者信息文件：patients.dat

为方便用户操作，程序采用文件夹作为存储单位：用户在主界面选择一个目录，系统会在该目录下读取或生成这两个文件。
读取流程：
用户通过菜单 “打开数据文件夹” 选择一个目录。
程序在该目录中依次尝试读取：
doctors.dat → 调用 DoctorInfortable::readDoctorFromFile()
patients.dat → 调用 PatInforTable::readPatFromFile()

如果任意一个文件成功读入，则刷新对应的模型（DoctorTableModel、BookingTableModel 等），并更新状态栏提示 “数据加载完成”；
若两个文件都不存在或读取失败，则提示 “无法加载数据文件”。

写入流程
用户通过菜单 “保存数据文件夹” 选择父一个目录。
之后填写父目录下建立的新文件夹名称
程序在该目录下生成/覆盖：
doctors.dat ← DoctorInfortable::saveDoctorToFile()
patients.dat ← PatInforTable::savePatToFile()
写入完成后，弹出提示 “保存成功”；若有任一写入失败，则提示 “保存失败”。

4 具体实现过程

4.1 程序搭建顺序

搭建遵循先确定主界面与需求，再逐步补齐数据与模型的原则。整个过程可以分为以下八个阶段：

4.1.1 主窗口界面搭建

首先完成主窗口 MainWindow 的设计。在 mainwindow.ui 中确定系统所需的菜单和功能入口，包括医生信息、预约门诊、查找、停诊管理和视图切换等。

4.1.2 对话框界面设计

在确认主窗口功能结构后，设计四个核心对话框的界面：

AddDoctor：用于医生信息录入和排班设置；

AddBooking：用于患者预约信息的新增；

SearchDialog：提供医生或患者信息的检索界面；

SuspendDialog：用于停诊记录的维护。

完成 .ui 文件的布局和控件命名，保证主窗口可以弹出这些对话框。

4.1.3 数据实体与集合搭建

在界面雏形完成后，定义系统的核心数据实体与全局集合：

Patient：患者及预约实体；

Doctor：医生个体及其名下预约集合；

PatInforTable：全院预约总表；

DoctorInfortable：全院医生总表。

数据可以在内存中被创建、修改和查询，并支持基本的文件读写。

4.1.4 文件夹级数据读写

将全局数据集合与主窗口对接，实现“目录为单位”的数据管理。用户选择文件夹后，系统会在其中读取或生成 doctors.dat 和 patients.dat 两个文件。确保系统能够作为数据容器使用，即保存后的数据可以在下次运行时重新加载。

4.1.5 对话框功能雏形实现

使四个对话框具备基本功能：

AddDoctor：收集医生信息并返回给主窗口；

AddBooking：实现科室与医生下拉框的联动，并提供基本的时间选择；

SuspendDialog：在内存中维护一组停诊记录；

SearchDialog：根据关键字生成初步的检索结果。

4.1.6 表格模型骨架建立

在数据实体和对话框逻辑具备后，开始建立表格模型，以驱动各视图的展示：

DoctorTableModel：医生列表模型；

BookingTableModel：全院预约桥梁模型；

DoctorBookingsTableModel：单医生预约模型；

SearchTableModel：搜索结果模型；

SuspendModel：停诊记录模型。

4.1.7 模块联动

在模型建立的基础上，进一步实现对话框与数据模型的双向联动：

AddDoctor：确认后将医生写入 DoctorInfortable，并刷新 DoctorTableModel

AddBooking：确认后将预约写入 PatInforTable，同时同步到对应 Doctor::d_patient，并刷新预约相关模型；

SearchDialog：将结果传递至 SearchTableModel，供表格展示；

SuspendDialog：更新 SuspendModel，供预约冲突检测调用。

4.1.8 一致性与 ui 美化

在最后阶段，对核心逻辑进行打磨，确保数据一致性和使用体验：

保证 PatInforTable（全局预约表）与 Doctor::d_patient（医生副本）始终同步；

在主界面显示上添加渐变背景，选择时的颜色区别等，使界面更美观

4.2 遇到的问题及解决方法

在程序开发过程中，遇到了一些问题与错误，主要包括以下几类：

4.2.1 提示信息乱弹

在自动调整预约时间时，程序会重复弹出提示框。经排查发现是业务逻辑在不同位置均触发了提示。

解决方法： 将提示逻辑集中管理，确保只在最终确认时弹出提示，从而避免重复触发。

4.2.2 对话框无法伸缩，使用普通布局出现错位

在设计对话框时，发现直接使用水平布局/垂直布局会造成控件错位，且窗口无法根据内容自动伸缩。

解决方法： 在合适位置添加 Spacer 占位符，使控件能够随窗口伸缩保持位置。

4.2.3 文件读写字符串编码问题

在保存和读取文件时，中文信息出现乱码。原因最后发现是未统一文件读写的编码格式。

解决方法： 将读写操作统一为 UTF-8 编码，确保医生姓名、症状等字段在不同平台下均能正常显示。

4.2.4 弹窗重复出现

在新建预约时，弹窗被触发两次。经检查发现，是在两个 ui AddBooking 与 MainWindow 内部均绑定了一次确认逻辑。

解决方法：注释掉重复绑定，仅保留 AddBooking 内的处理逻辑，保证弹窗只出现一次。

4.2.5 按钮点击无效

部分按钮点击后没有响应，原因最后排查出是在 Qt Designer 中多次创建了同名的 action，系统自动将新命名改为 _2 后导致槽函数未能绑定。

解决方法： 清理重复的 action，保证每个控件名称唯一，并重新绑定到对应槽函数。

4.2.6 图片无法显示

在界面中加载贴图时，发现贴图无法正常显示。原因最终确认为格式不匹配

解决方法：将资源统一转换为 .png 格式，并在界面加载时确保路径和资源

文件正确引用。

5 测试报告

总览功能表如下:

功能模块	测试内容	预期结果	实际结果
5.1 主界面	程序启动后主界面加载；左下角显示统计信息；菜单栏是否完整	左下角统计显示正常，菜单栏功能项完整可见	通过
5.2 医生管理	点击“视图→显示医生”，进入医生表格	医生表格正确显示	通过
	点击“医生信息→新增医生”，输入完整信息后确认	新医生即时加入表格	通过
	输入重复医生编号	弹出提醒，拒绝保存	通过
	点击“医生信息→编辑医生信息”，修改后保存	表格数据更新，姓名和编号保持不可改	通过
	点击“医生信息→删除医生信息”，确认删除	医生及名下预约一并删除	通过
	点击“视图→显示医生预约数据”，选择医生	显示该医生名下预约，按时间顺序排列	通过
5.3 预约管理	点击“视图→显示预约”，进入预约表格	全部预约信息正确显示	通过
	点击“预约门诊→新增预约”，填写完整信息后确认	新预约写入表格，并同步至对应医生名下	通过
	医生层面：预约时间冲突	自动跳转到最近可用的时间	通过
	病人层面：相同时间重复预约	弹出提醒，拒绝保存	通过
	输入信息不全	弹出提醒	通过
	不同病人使用相同身份证号	弹出提醒，拒绝保存	通过
	点击“预约门诊→修改预约”，选择预约后修改	对话框预填原数据，修改后保存成功	通过
	点击“预约门诊→删除预约”	选定预约删除成功	通过
5.4 查找系统	点击“查找→查找预约记录/查找医生”，选择字段并输入条件	弹出查找对话框，结果显示在表格中	通过
	点击“搜索”	显示匹配结果及匹配条数	通过

功能模块	测试内容	预期结果	实际结果
5.5 停诊管理	点击“管理停诊”，进入停诊对话框	可新增停诊记录，保存后停诊表格中显示	通过
	在停诊日期尝试预约	系统自动跳到下一个可行时间	通过
5.6 文件保存与打开	点击“文件→保存文件”，选择目录并输入文件夹名	成功保存为新建命名文件夹下 doctors.dat 和 patients.dat 文件	通过
	点击“文件→打开文件”，选择数据文件夹	成功读取并恢复医生与患者数据	通过

5.1 主界面

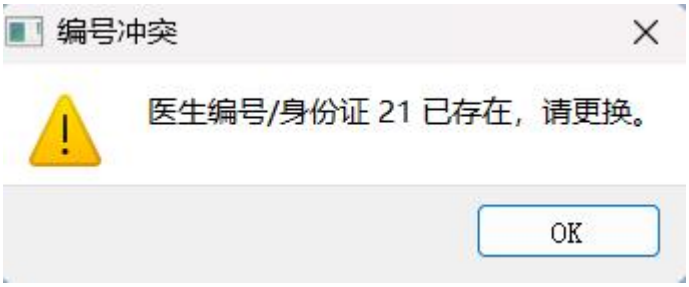
姓名	身份证号	性别	年龄	预约日期	预约时段	医生	科室	联系电话	症状描述
病人2号	2222	女	26	2025-09-16	15:30-16:00	预医生	预防保健科	12134	我也找预
病人3号	3333	男	6	2025-09-20	09:00-09:30	内科医生	内科	16782324	找内科
病人4号	44	女	28	2025-09-24	10:00-10:30	内科医生	内科	112235	我找内科
病人5号	555	男	35	2025-09-20	15:00-15:30	内科医生	内科	1789234	找内科
病人6号	666	男	26	2025-09-18	09:00-09:30	全医生	全科医疗科	17892845	找全科
病人8号	88	男	25	2025-09-18	09:30-10:00	全医生	全科医疗科	1323545	全科
病人	23	女	12	2025-09-24	11:00-11:30	内科医生	内科	123	完全
病人	23	女	12	2025-09-16	16:30-17:00	全医生	全科医疗科	123	为我
病人2	2334	男	13	2025-09-18	08:00-08:30	预医生	预防保健科	123	是

左下角统计显示正常，菜单栏完整

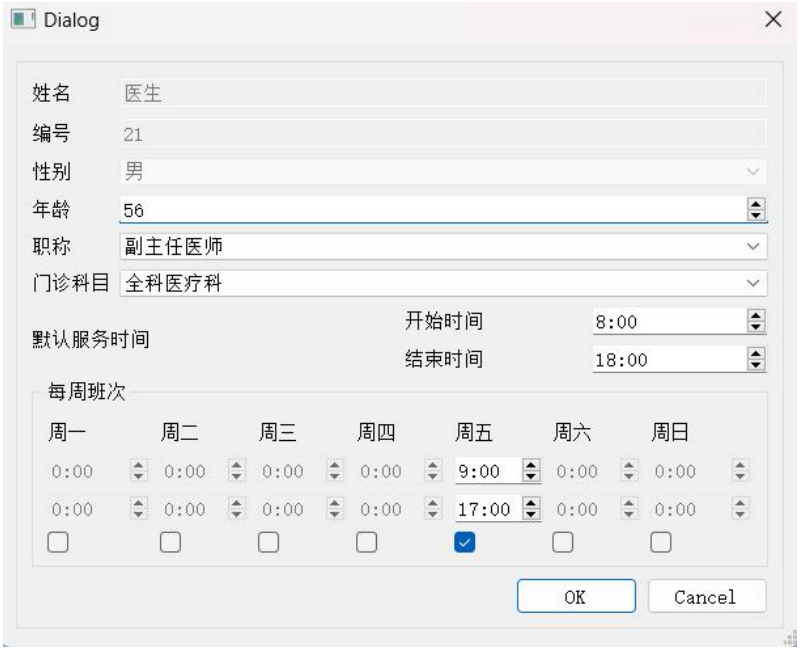
5.2 医生管理

点击“视图”“显示医生”会进入医生表格
点击菜单 “医生信息”“新增医生”，弹出 AddDoctor 对话框。
在对话框中输入姓名、科室、职称、上下班时间等信息，点击确认后，医生表格即时更新。

如果输入的医生编号与已有编号重复，会弹出提醒

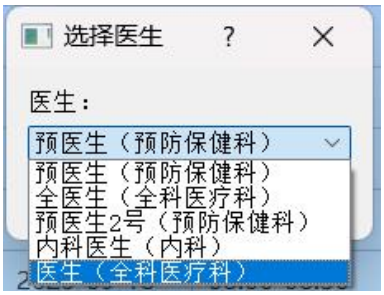


选中医生后可以通过点击“医生信息”“编辑医生信息”或“删除医生信息”进行修改和删除

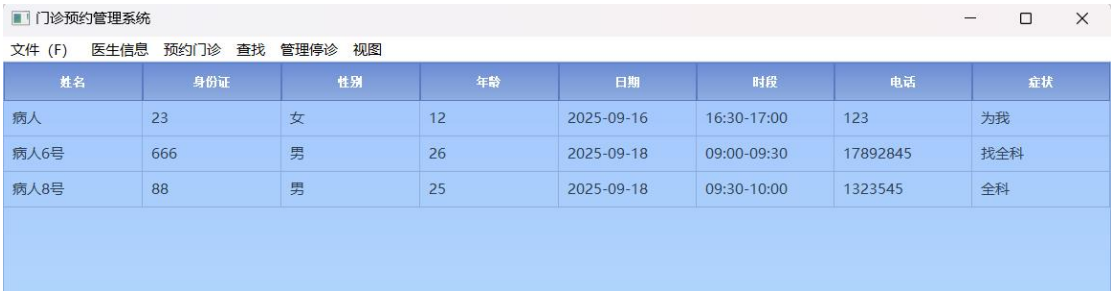


修改时已有信息会预填，但姓名和编号不可改

点击“视图”“显示医生预约数据”会弹出选择医生 ComboBox



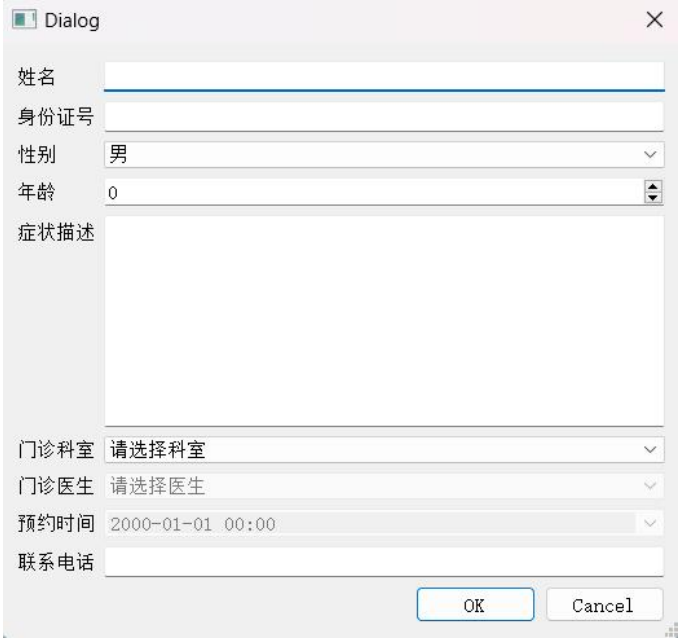
选择后会按预约时间顺序显示该医生名下的预约汇总



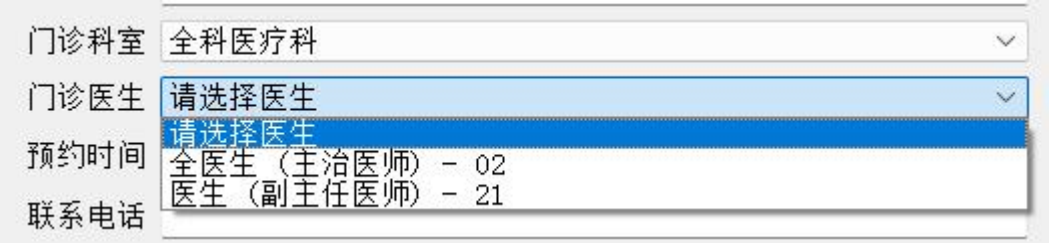
5.3 预约管理

点击“视图”“显示预约”会进入预约表格
点击“预约门诊”“新增预约”
若此时无医生信息，会弹出提示

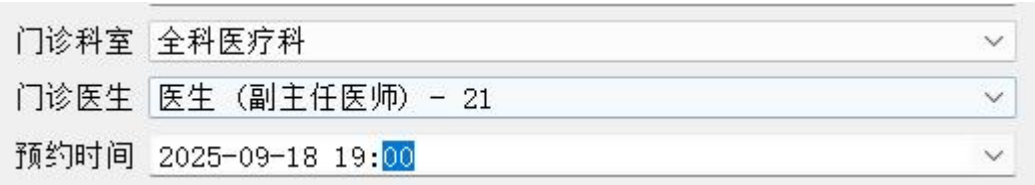
若已有医生信息，会弹出 AddBooking 对话框

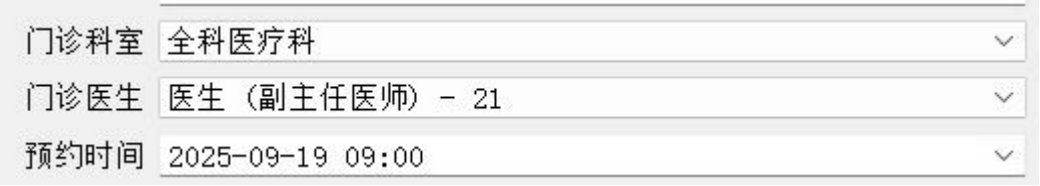


科室 ComboBox 下拉框中为已有医生的对应科室
选择科室后可启用选择医生 ComboBox

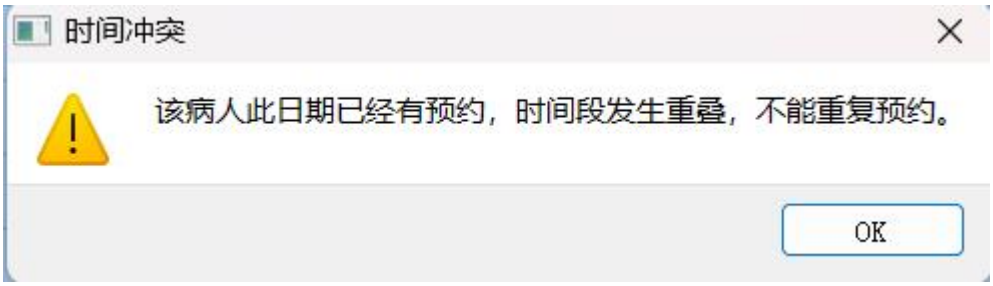


对医生层面的预约时间冲突，若所选时间不可用，在确认/回车后会自动跳转到下一个最近的可用时间



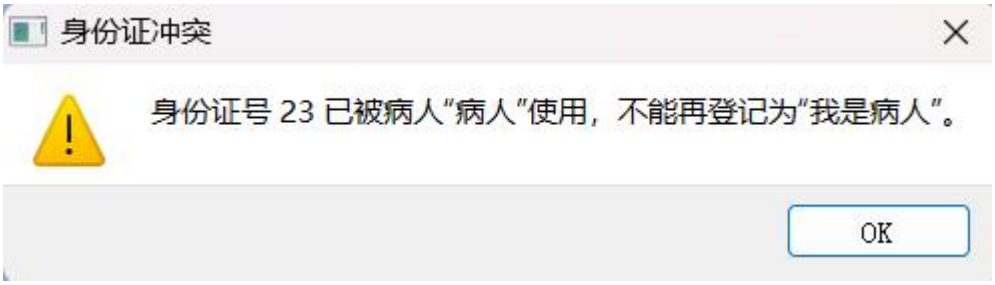


对病人层面的预约时间冲突，会在确认后弹出提醒



对信息不全的预约也会弹出提醒

同时会检测是否有不同病人使用了相同身份证号



点击“预约门诊”“修改预约”会弹出 ComboBox 进行要修改的预约的选择

修改时同样会预填原数据

点击“预约门诊”“删除预约”可进行删除

5.4 查找系统

点击“查找”“查找预约记录”或“查找医生”会弹出查找对话框，可以选择具体的查找项目

下图为查询医生信息，查询预约同理

点击“搜索”后会显示表格及匹配条数

5.5 停诊管理

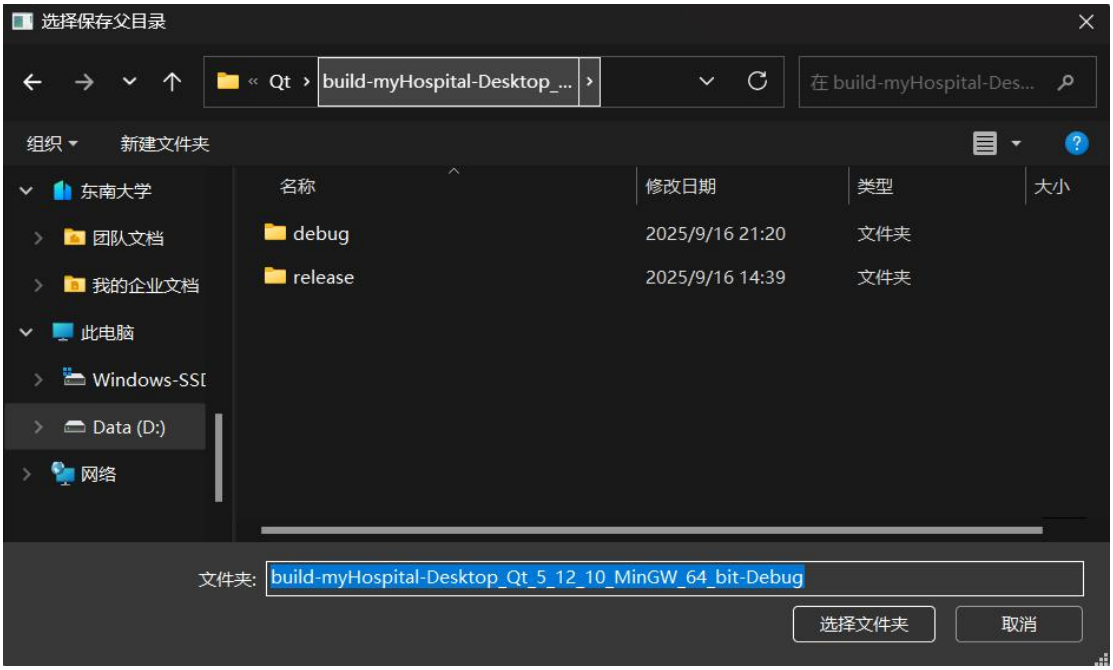
点击“管理停诊”后会弹出 SuspendDialog

保存后会在 diaglog 中显示

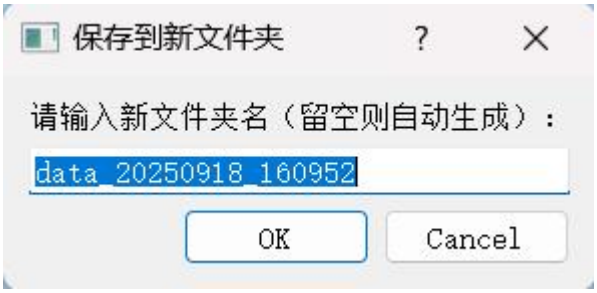
若是在这个时间选择预约该医生，同样会直接弹到下一个可行时间

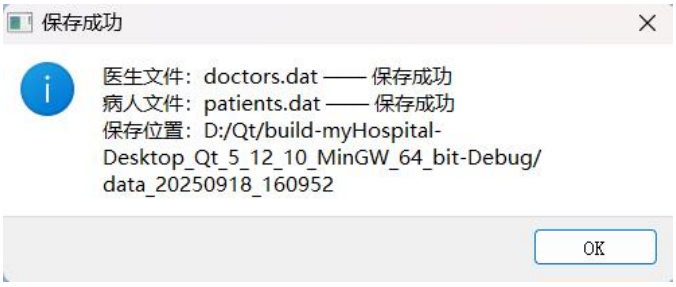
5.6 文件的保存与打开

点击“文件”“保存文件”会弹出父目录选择

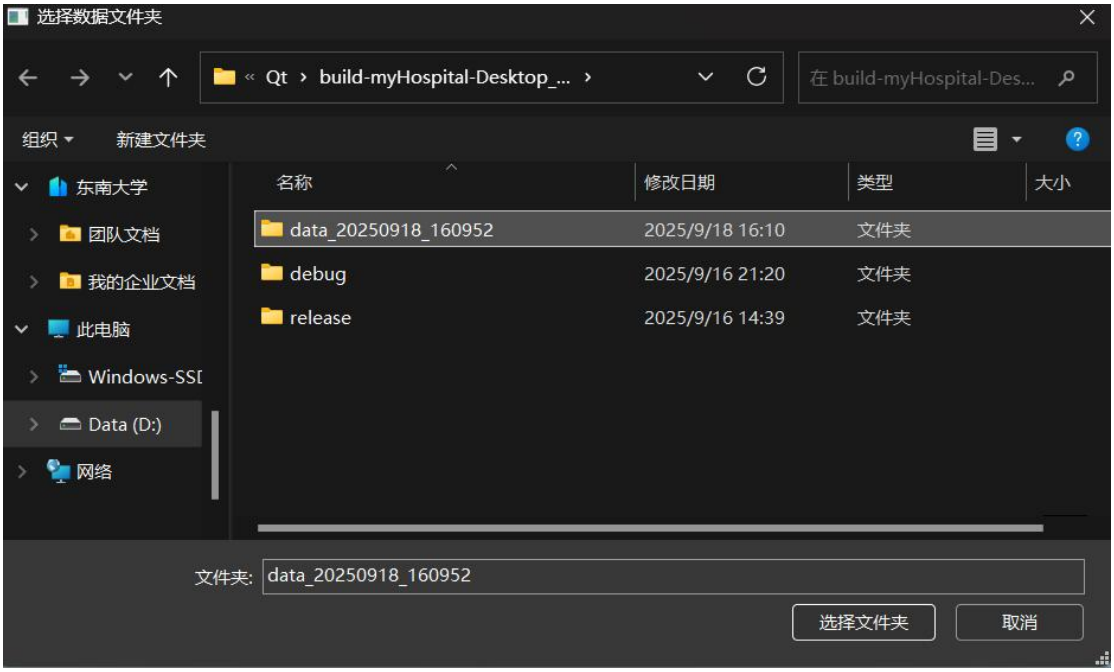


选择后会要求输入文件夹名





点击“文件”“打开文件”可以选择数据文件夹（包含 doctor.dat 和 patient.dat 的文件夹）



6 心得体会和相关建议（课程小结和对课程改进的建议）

6.1 心得体会

这次的 Qt 课程设计让我收获颇丰。在整个项目的实现过程中，我巩固了 C++ 的面向对象编程思想，更深入掌握了 Qt 框架的实际应用，包括信号与槽机制、界面布局管理、模型与视图框架等特性。通过系统地完成从界面设计、数据模型构建，到文件读写与业务逻辑实现的全过程，我对 Qt 的使用方法和整体软件开发流程有了更加全面的认识。

在实现过程中，我遇到了不少挑战。例如，如何理清复杂的类关系，保证医生表、预约表、停诊表三者之间的数据一致性；如何通过模型-视图架构让表格既能展示信息，又能实时增删改；以及如何保证用户界面友好。这些问题迫使我不断查阅文档，参考在线资料，并通过调试与实践逐步解决。最终，我不仅完成了一个功能较为完整的系统，也提升了自己在架构设计和问题解决上的能力。

本次大作业也让我深刻体会到细节处理的重要性。比如在预约模块中，如果身份证号重复、预约时间冲突或遇到医生停诊，系统都需要给出清晰的提示；在文件读写时，如果不注意编码格式，就会出现中文乱码；在界面设计中，

若控件布局没有合理使用 Spacer，对话框就无法伸缩。这些细节问题虽然在整体功能上看似“不重要”，但如果不加优化，系统就无法真正贴合用户需求。通过一遍遍调试与修改，我体会到“让系统运行”只是基础，更重要的是“让系统好用”。

同时，我在这次课程设计中也逐渐养成了良好的注释习惯。一开始，注释只是帮助我理解老师给的示例；到后来，随着自己逐步设计和实现代码，注释变成了我与自己思路的对话。在遇到复杂的逻辑时，注释能让我快速回溯设计意图，也帮助我在调试时更高效地定位问题。

总的来说，这次大作业让我真正体会到了一个完整软件开发的复杂性与成就感。它不仅锻炼了我的编程能力和工程实践能力，也让我明白：一个合格的程序不仅要实现功能，更要考虑用户体验和健壮性。未来我会继续深入学习 Qt 和 C++，提升自己在项目开发中的综合能力

6.2 课程改进建议

总体而言我认为是很好的课程，只是最后验收的时候可能会人比较多，因此建议之后可以进行更小班的教学，如一个班 20 人左右可能是比较合适的